

GenAI Cohort - Class-14

Summarize this . as why we use , graphDB , need ? what is diff bw sqldb , nosql and graphdb

Absolutely. The main idea of your image is **why GraphDB exists and when it is better than SQL/NoSQL**.

1. Why do we need a Graph Database?

The key problem is **relationships**.

Consider this example from your diagram:

Alice —likes→ CCD

Alice —hates→ HC

Jane —likes→ Hot Coffee

Jane —likes→ CCD

Jane —hates→ Rest

Alice —likes→ Jane

Here, we don't just care about the **data** (Alice , Jane , CCD , Coffee).

We care heavily about **how things are connected**.

For example:

"Find people who like the same coffee as Jane."

That's a **relationship/traversal problem**.

Graph databases are designed specifically for this kind of data.

2. Three different database approaches

Think of them like this:

Database	Main focus	Data structure
SQL	Structured data + transactions	Tables → Rows → Columns

Database	Main focus	Data structure
NoSQL	Flexible/unstructured data	Documents / Key-Value / Collections
GraphDB	Relationships between data	Nodes → Relationships → Properties

3. SQL Database

In SQL, we normally represent the above data using tables.

For example:

Users

id	name
1	Alice
2	Jane

Products

id	name
101	CCD
102	Hot Coffee

Likes

user_id	product_id
1	101
2	101
2	102

Now suppose you want:

Find people who like what Jane likes.

You need to **JOIN** multiple tables.

```
SELECT u.name
FROM Users u
JOIN Likes l ON u.id = l.user_id
JOIN Products p ON l.product_id = p.id
WHERE p.id IN (
    SELECT product_id
    FROM Likes
    WHERE user_id = 2
);
```

The database is essentially navigating through **foreign keys and JOINS**.

SQL is excellent when:

- Data has a well-defined structure
 - Relationships are relatively straightforward
 - You need strong transactions
 - You need reporting/aggregation
 - You frequently query rows and columns
-

4. NoSQL Database

A document database such as MongoDB might store something like:

```
{
  name: "Jane",
  likes: ["CCD", "Hot Coffee"],
  hates: ["Rest"]
}
```

This is much more flexible.

You don't necessarily need a rigid table structure.

That's why NoSQL is useful when:

- Data structure changes frequently
- Data is document-oriented
- You want flexible schemas
- You want to store JSON-like objects

But there's a problem.

Suppose your application becomes:

Alice → likes → CCD
 Alice → follows → Bob
 Bob → worksAt → Google
 Google → owns → YouTube
 YouTube → created → Video
 Video → watchedBy → Jane
 Jane → likes → CCD

Now the **connections themselves become the important part.**

Representing and querying these deeply connected relationships using documents can become complicated.

5. Graph Database

This is where GraphDB comes in.

Instead of thinking:

TABLE
 ↓
 ROW
 ↓
 COLUMN

we think:

NODE
 ↓
 RELATIONSHIP
 ↓
 NODE

For example:

(Alice) —[:LIKES]—> (CCD)

(Alice) —[:LIKES]—> (Jane)

(Jane) —[:LIKES]—> (Hot Coffee)

A graph consists mainly of:

Node

Represents an entity.

Alice
Jane
CCD
Hot Coffee

Relationship

Represents the connection.

Alice —LIKES—> CCD
Alice —LIKES—> Jane
Jane —HATES—> Rest

Properties

Additional information.

```
Alice {  
  age: 22  
}
```

Alice —LIKES {since: 2025}—> CCD

6. The biggest difference

This is the **most important thing to remember**:

SQL

Data → Tables
Relationships → JOINS

NoSQL

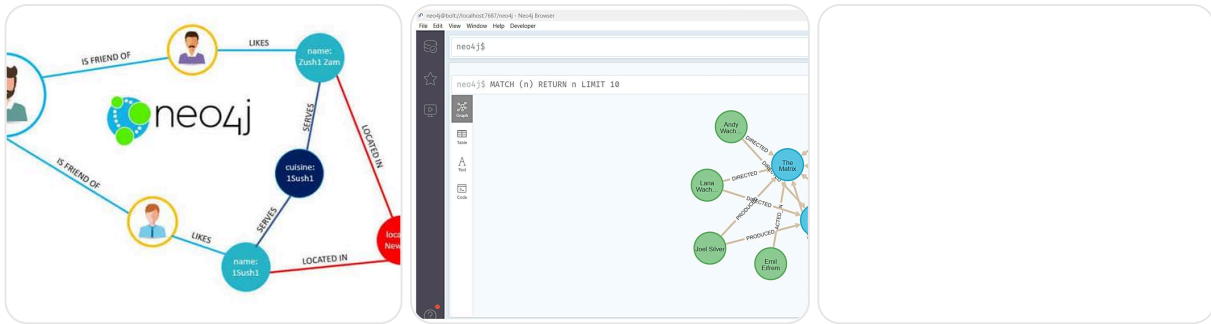
Data → Documents
Relationships → Usually embedded/referenced manually

GraphDB

Data → Nodes
Relationships → First-class citizens

That's why GraphDB is particularly powerful when **relationships are the core of the problem**.

7. Why Neo4j?



Neo4j is one of the most popular graph databases.

It stores:

Nodes

Relationships

Properties

And we query it using **Cypher**.

For example:

```
MATCH (alice:User {name: "Alice"})-[:LIKES]->(coffee)
RETURN coffee
```

This literally says:

Find the **User** named Alice, follow a **LIKES** relationship, and return the connected coffee.

Notice how naturally the query represents the graph.

8. Why Cypher feels different from SQL

SQL asks:

Which rows should I retrieve?

Cypher asks:

What pattern exists in my graph?

For example:

```
(Alice)-[:LIKES]->(CCD)
```

means:

Alice

|

| LIKES



CCD

You can extend the pattern:

(Alice)-[:LIKES]->(CCD)<-[:LIKES]-(Jane)

Meaning:

Find Alice and Jane who both like the same CCD.

This is where GraphDB becomes very powerful.

9. GraphDB and "no fixed schema"

Your image says:

There is no fixed schema

This is broadly referring to **schema flexibility**.

Unlike traditional SQL tables where you normally define columns beforehand:

User

id

name

email

age

a graph can have nodes with different properties:

Alice

```
{
  name: "Alice",
  age: 22
}
```

and another node:

Company

```
{
  name: "Google",
  location: "California",
  employees: 100000
}
```

You don't have to force every node to have exactly the same properties.

Important: Neo4j isn't literally "schema-less" in every sense. Neo4j supports **indexes and constraints**, so the better description is **schema-flexible**.

10. When should we use GraphDB?

This is the key question.

Use GraphDB when your application has **complex and highly connected relationships**.

Examples

Social network

User → follows → User

User → likes → Post

User → comments → Post

Recommendation system

User → bought → Product

Product → belongsTo → Category

User → likes → Category

Fraud detection

Account → uses → Device

Device → usedBy → Account

Account → transfersTo → Account

You can discover suspicious connection patterns.

Knowledge Graph / AI

Einstein → workedAt → Princeton

Einstein → developed → Theory of Relativity

Theory of Relativity → relatedTo → Physics

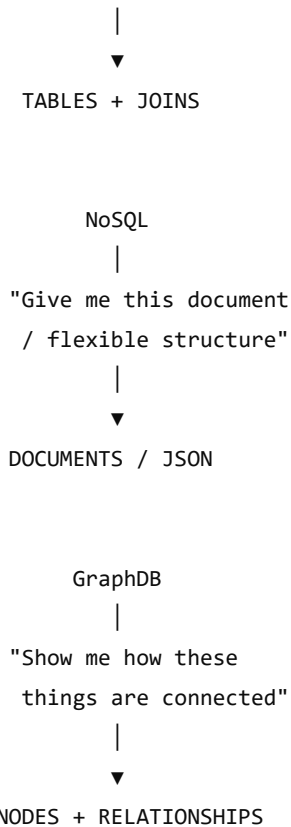
Physics → fieldOf → Science

This is especially useful for **knowledge graphs and RAG systems**.

11. Simple mental model

Remember this:

```
SQL
|
"Give me the rows
matching these conditions"
```

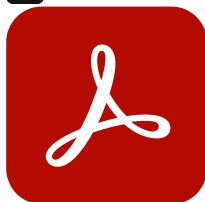



One-line difference

SQL is optimized around structured tabular data, NoSQL around flexible document/key-value data, while GraphDB is optimized around connected data and traversing relationships.

5 key points to remember

1. SQL → Tables, rows, columns, JOINS.
2. NoSQL → Flexible documents/data structures.
3. GraphDB → Nodes + Relationships + Properties.
4. GraphDB shines when relationships are the most important part of the data.
5. Neo4j uses Cypher to describe and find graph patterns naturally.



Convert to PDF

Now Explain Cypher Queries , basic one's like how to create something , delete something , make relationships

and query ..

Absolutely. Let's learn **Cypher from the ground up** using the same `Alice`, `Jane`, `CCD`, `Coffee` example.

The easiest way to understand Cypher is:

Cypher describes patterns in a graph.

1. Basic Cypher Structure

A Cypher query often looks like:

```
MATCH (node)
WHERE condition
RETURN node
```

Think of it as:

```
MATCH → Find something
WHERE → Filter it
RETURN → Give me the result
```

But before querying, let's understand how we **create** data.

2. Creating a Node

Suppose we want to create Alice.

```
CREATE (alice:User {name: "Alice"})
```

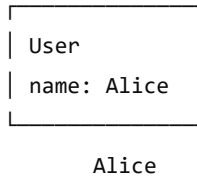
Break it down:

```
CREATE
  ↓
(alice:User {name: "Alice"})
  ↓      ↓      ↓
variable label  property
```

```
(alice:User {name: "Alice"})
```

- `alice` → variable name
- `User` → label
- `name: "Alice"` → property

So we created:



3. Create Multiple Nodes

You can create several nodes in one query:

```
CREATE
  (alice:User {name: "Alice"}),
  (jane:User {name: "Jane"}),
  (ccd:Coffee {name: "CCD"}),
  (hotCoffee:Coffee {name: "Hot Coffee"})
```

Now our graph contains:

Alice Jane

CCD Hot Coffee

But currently, **nothing is connected**.

That's where relationships come in.

4. Creating a Relationship

Suppose:

Alice likes CCD.

We can create:

```
CREATE
  (alice:User {name: "Alice"})
  -[:LIKES]->
  (ccd:Coffee {name: "CCD"})
```

This creates both nodes and the relationship.

Graph:

Alice —LIKES—> CCD

The relationship syntax is:

```
-[:RELATIONSHIP_TYPE]->
```

For example:

```
-[:LIKES]->
-[:HATES]->
-[:FOLLOWS]->
-[:WORKS_AT]->
-[:FRIEND_OF]->
```

5. Creating a Relationship Between Existing Nodes

Usually, the nodes already exist.

For example, Alice and Jane already exist.

We first find them using `MATCH`.

```
MATCH
  (alice:User {name: "Alice"}),
  (jane:User {name: "Jane"})
CREATE
  (alice)-[:LIKES]->(jane)
```

Now:

Alice ———LIKES——> Jane

Notice something important:

```
MATCH
```

doesn't create anything.

It **finds existing data**.

Then:

```
CREATE
```

creates the relationship.

6. Creating More Relationships

Let's say:

```
Alice likes CCD
Jane likes CCD
Jane likes Hot Coffee
Jane hates Rest
```

We could write:

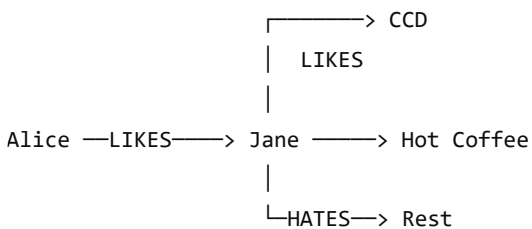
MATCH

```
(alice:User {name: "Alice"}),
(jane:User {name: "Jane"}),
(ccd:Coffee {name: "CCD"}),
(hotCoffee:Coffee {name: "Hot Coffee"}),
(rest:Coffee {name: "Rest"})
```

CREATE

```
(alice)-[:LIKES]->(ccd),
(jane)-[:LIKES]->(ccd),
(jane)-[:LIKES]->(hotCoffee),
(jane)-[:HATES]->(rest)
```

Our graph becomes:



7. MATCH — Finding Nodes

Now suppose we want to find Alice.

```
MATCH (alice:User {name: "Alice"})
RETURN alice
```

Meaning:

MATCH

↓

Find a User whose name is Alice

↓

RETURN

↓

Give me Alice

8. MATCH Without a Label

You can also do:

```
MATCH (person {name: "Alice"})
RETURN person
```

This searches for a node with:

```
name = "Alice"
```

But using labels is generally better:

```
MATCH (person:User {name: "Alice"})  
RETURN person
```

Because we're explicitly saying:

Search among **User** nodes.

9. Return Only a Property

We don't always need the entire node.

```
MATCH (person:User {name: "Alice"})  
RETURN person.name
```

Result:

Alice

You can return multiple properties:

```
MATCH (person:User {name: "Alice"})  
RETURN person.name, person.age
```

10. MATCH a Relationship

This is where Cypher becomes really interesting.

Suppose:

Alice —LIKES—> CCD

We can query:

```
MATCH (alice:User)-[:LIKES]->(coffee)  
RETURN coffee
```

Read it almost like English:

Find a **User** who **LIKES** something, and return that something.

Result:

ccd

11. Start From Alice

Suppose there are many users, but we specifically want Alice's liked items.

```
MATCH (alice:User {name: "Alice"})-[:LIKES]->(coffee)
RETURN coffee
```

Graph pattern:

```
(alice) —LIKES—> (coffee)
```

Cypher follows that relationship.

12. Find Who Alice Likes

Relationships don't have to point only toward products.

Suppose:

```
Alice —LIKES—> Jane
```

Query:

```
MATCH (alice:User {name: "Alice"})-[:LIKES]->(person:User)
RETURN person
```

Result:

```
Jane
```

13. Relationship Direction Matters

These are different:

```
(alice)-[:LIKES]->(jane)
```

means:

```
Alice —LIKES—> Jane
```

Whereas:

```
(alice)<-[:LIKES]-(jane)
```

means:

```
Alice <—LIKES— Jane
```

You can also ignore direction:

```
(alice)-[:LIKES]-(jane)
```

This means:

Find a **LIKES** relationship between them regardless of direction.

14. WHERE — Filtering

Suppose we have:

Alice
Jane
Bob
Rahul

We can find users older than 20:

```
MATCH (user:User)
WHERE user.age > 20
RETURN user
```

Think:

```
MATCH → find users
WHERE → filter users
RETURN → show result
```

You can also write the condition directly inside the pattern:

```
MATCH (user:User {name: "Alice"})
RETURN user
```

For more complex conditions, **WHERE** is useful.

15. DELETE a Node

Suppose we want to delete Alice.

```
MATCH (alice:User {name: "Alice"})
DELETE alice
```

But there is an important problem.

If Alice has relationships:

Alice —LIKES—> CCD

Neo4j normally won't let you simply delete Alice because the relationships still exist.

You need:

```
MATCH (alice:User {name: "Alice"})
DETACH DELETE alice
```


Difference

DELETE

→ Delete the node only if it has no relationships.

DETACH DELETE

→ Delete the node **and all relationships connected to it**.

Be careful with `DETACH DELETE`.

16. Delete a Relationship

Suppose:

Alice —LIKES→ CCD

We want to remove only the `LIKES` relationship.

```
MATCH (alice:User {name: "Alice"})-[r:LIKES]->(ccd:Coffee {name: "CCD"})
DELETE r
```

Here:

`r`

is the relationship variable.

So:

```
(alice)-[r:LIKES]->(ccd)
```

means:

```
alice
  |
  | r = relationship
  |
  ▼
ccd
```

Then:

```
DELETE r
```

deletes the relationship but **keeps both nodes**.

17. Updating a Node — SET

Suppose Alice's age is 21 and we want to change it to 22.

```
MATCH (alice:User {name: "Alice"})
SET alice.age = 22
RETURN alice
```

SET is used to add or update properties.

For example:

```
SET alice.city = "Delhi"
```

Now Alice has:

```
{
  name: "Alice",
  age: 22,
  city: "Delhi"
}
```

18. Updating a Relationship

Relationships can also have properties.

Suppose:

```
(alice)-[r:LIKES]->(ccd)
```

We can add:

```
SET r.since = 2025
```

So now:

```
Alice —LIKES {since: 2025}—> CCD
```

19. MERGE — Very Important

One problem with **CREATE** is that it always creates.

For example:

```
CREATE (alice:User {name: "Alice"})
```

Run this twice:

```
Alice
Alice
```

You may accidentally create duplicates.

MERGE is used when you want:

Find it if it exists, otherwise create it.

```
MERGE (alice:User {name: "Alice"})
RETURN alice
```

If Alice exists:

Find Alice

If Alice doesn't exist:

Create Alice

This is extremely useful when inserting data where duplicates should be avoided.

20. MERGE for Relationships

Suppose we want Alice to like CCD.

```
MATCH
  (alice:User {name: "Alice"}),
  (ccd:Coffee {name: "CCD"})
MERGE
  (alice)-[:LIKES]->(ccd)
```

If the relationship already exists:

Don't create another one.

If it doesn't exist:

Create it.

21. A Very Important Query Pattern

You will frequently write this:

```
MATCH (a:User {name: "Alice"})-[:LIKES]->(coffee)
RETURN coffee
```

Understand the pattern:

```
MATCH
  |
  ▼
(Alice) —LIKES—> (coffee)
  |               |
  known          unknown
```



Cypher lets you **capture unknown nodes/relationships into variables**.

22. Multi-Hop Queries

This is one of the biggest advantages of GraphDB.

Suppose:

```

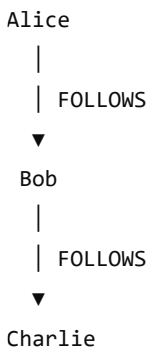
Alice —FOLLOWS—> Bob
Bob —FOLLOWS—> Charlie
Charlie —WORKS_AT—> Google
  
```

We can query:

```

MATCH (alice:User {name: "Alice"})
      -[:FOLLOWS]->
        (person:User)
      -[:FOLLOWS]->
        (friend:User)
RETURN friend
  
```

We're traversing:



This is where Cypher becomes much more powerful than simply thinking in terms of tables.

23. Variable-Length Relationships

Suppose you want to find people Alice can reach through **1 to 3 FOLLOWS** relationships.

You can write:

```
MATCH (alice:User {name: "Alice"})  
      -[:FOLLOWS*1..3]->  
      (person:User)  
RETURN person
```

Meaning:

*1..3

= traverse between **1 and 3 relationships**.

So Cypher can naturally express graph traversal.

24. The Core Cypher Commands

For now, remember these:

Cypher	Purpose
CREATE	Create nodes/relationships
MATCH	Find existing nodes/relationships
WHERE	Filter results
RETURN	Return results
SET	Add/update properties
DELETE	Delete nodes/relationships
DETACH DELETE	Delete node + connected relationships
MERGE	Find or create
REMOVE	Remove a property/label

25. Complete Example

Let's build a small graph from scratch.

Step 1 — Create users

```
CREATE
  (alice:User {name: "Alice", age: 22}),
  (jane:User {name: "Jane", age: 23})
```

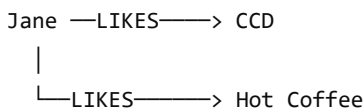
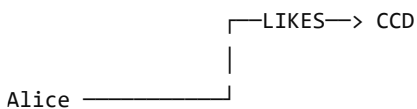
Step 2 — Create coffee

```
CREATE
  (ccd:Coffee {name: "CCD"}),
  (hotCoffee:Coffee {name: "Hot Coffee"})
```

Step 3 — Connect them

```
MATCH
  (alice:User {name: "Alice"}),
  (jane:User {name: "Jane"}),
  (ccd:Coffee {name: "CCD"}),
  (hotCoffee:Coffee {name: "Hot Coffee"})
CREATE
  (alice)-[:LIKES]->(ccd),
  (jane)-[:LIKES]->(ccd),
  (jane)-[:LIKES]->(hotCoffee)
```

Now:



Step 4 — Find what Jane likes

```
MATCH (jane:User {name: "Jane"})-[:LIKES]->(coffee)
RETURN coffee.name
```

Result:

```
CCD
Hot Coffee
```

Step 5 — Find people who like CCD

```
MATCH (person:User)-[:LIKES]->(coffee:Coffee {name: "CCD"})
RETURN person.name
```

Result:

Alice

Jane

And this last query is a perfect example of **why GraphDB is useful**:

We describe the relationship pattern we want, and Neo4j finds the matching paths in the graph.

The Mental Model You Should Keep

Whenever you see Cypher, think:

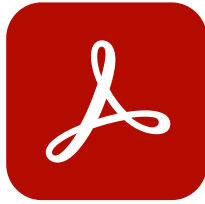
```
MATCH
↓
What pattern
am I looking for?
↓
WHERE
↓
Should I filter?
↓
RETURN
↓
What do I want
as output?
```

And for modifying data:

```
CREATE → Create something
MERGE → Find or create
SET → Update something
DELETE → Remove something
```

5 key points

1. **MATCH** = find existing graph data.
2. **CREATE** = create nodes or relationships.
3. **MERGE** = create only if the pattern doesn't already exist.
4. **SET** / **DELETE** = modify or remove data.
5. Cypher's biggest strength is describing relationship patterns and traversing multiple connected nodes.



Convert to PDF

Now Explain these queries -> 1. CREATE (aliceUser { name : "Alice" , age: 25, city: "New York"}); 2. CREATE (systemDesign: Interest) {name : "System Design"} 3. MATCH (i:Interest) return i; 4. MATCH (i:Interest), (u:User) return i,u; 5. MATCH (bob:User {name: "Bob"}), (systemDesign:Interest {name:"System Design"}) RETURN bob,systemDesign 6. MATCH (bob:User {name: "Bob"}), (systemDesign:Interest {name:"System Design"}) CREATE (bob)-[:INTERESTED_IN]->(systemDesign) RETURN bob,systemDesign 7. MATCH u=()-[:!INTERESTED_IN]->(a:Interest) return u 8. MATCH u=()-[:!INTERESTED_IN]->(a:Interest) return a; . Explain them In Flow and with proper flow diagrams

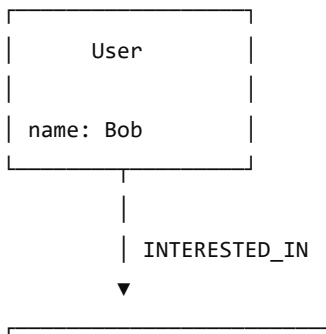
Yes. These 8 queries are a **very good sequence for understanding Neo4j**, because they take us from:

Create Nodes → Find Nodes → Find Multiple Nodes → Create Relationship → Traverse Relationship → Return Different Parts of the Pattern.

I'll explain them as one continuous flow so you understand **what Neo4j is doing internally conceptually**.

First: Understand the Graph We Are Building

By the end of these queries, we want this:



Interest
name: System Design

Later, we query this relationship.

1. CREATE Alice

```
CREATE (aliceUser {
  name: "Alice",
  age: 25,
  city: "New York"
});
```

What is happening?

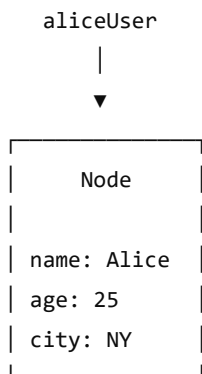
CREATE tells Neo4j:

Create a new node.

The pattern is:

```
(aliceUser { properties })
```

Break it down:



Important

aliceUser is a **variable name**.

It is NOT the node's name.

So:

```
(aliceUser {name: "Alice"})
```

means:

```
variable = aliceUser
property = name → Alice
```

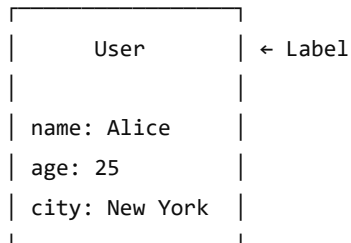
Also notice that there is **no** `:User` label here.

So this creates an **unlabeled node**.

If you want Alice to be a `User`, normally write:

```
CREATE (aliceUser:User {
  name: "Alice",
  age: 25,
  city: "New York"
});
```

Then:



2. CREATE System Design Interest

Your query is written as:

```
CREATE (systemDesign: Interest) {
  name: "System Design"
}
```

There is a **syntax issue** here.

The correct Cypher syntax is:

```
CREATE (systemDesign:Interest {
  name: "System Design"
});
```

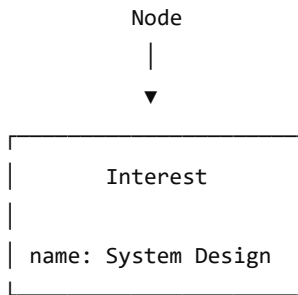
Everything describing the node must be **inside the parentheses**.

Understand the structure

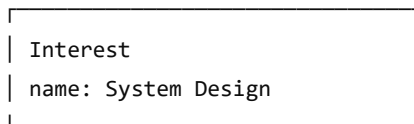
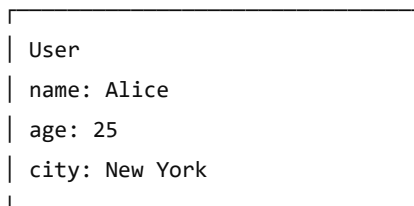
```
(systemDesign:Interest {name: "System Design"})
|           |           |
```

variable label property

So:



After Query #1 and #2:



They are not connected yet.

3. MATCH an Interest

MATCH (i:Interest)

RETURN i;

This is our first **query/read operation**.

Think:

MATCH

↓

Find something

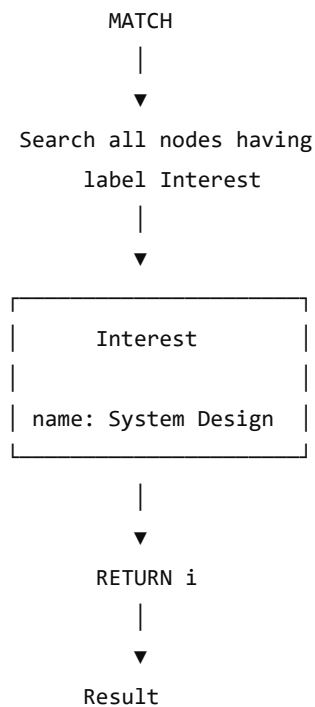
↓

RETURN

↓

Give me what you found

Step-by-step flow



What is `i`?

`(i:Interest)`

Here:

`i`

|

└ variable representing the matched node

So:

`RETURN i`

means:

Return the entire node represented by `i`.

4. MATCH Interest AND User

`MATCH (i:Interest), (u:User)`

`RETURN i, u;`

This query is extremely important for understanding Cypher.

We have **two patterns**:

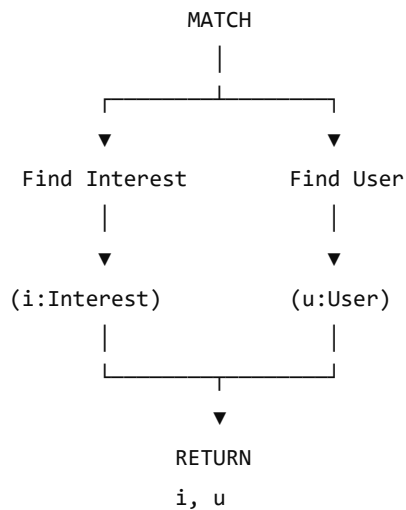
(i:Interest)

and

(u:User)

They are separated by a comma.

Flow



Suppose the database contains:

Alice → User

Bob → User

System Design → Interest

Then Neo4j can return combinations of the matched **Interest** and **User** nodes.

Important concept

There is **no relationship pattern** here.

You're simply saying:

Find nodes labeled **Interest** and nodes labeled **User**.

This is different from:

`MATCH (u:User)-[:INTERESTED_IN]->(i:Interest)`

which specifically asks for **connected User → Interest patterns**.

5. Find Bob and System Design

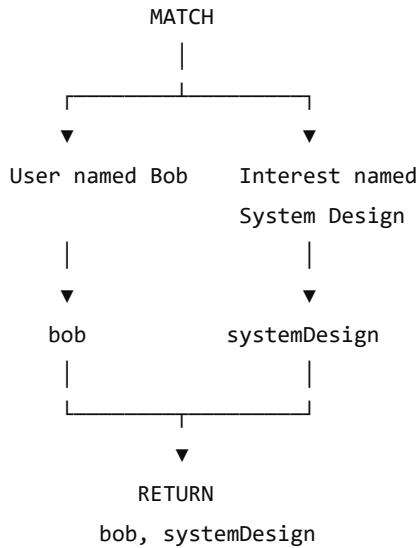
MATCH

```
(bob:User {name: "Bob"}),
(systemDesign:Interest {name: "System Design"})
```

RETURN bob, systemDesign;

Now we're being much more specific.

We have:



The important difference from #4 is the **property filtering** inside the node pattern.

```
(bob:User {name: "Bob"})
```

means:

Find a node with label `User` whose `name` property is `"Bob"`.

And:

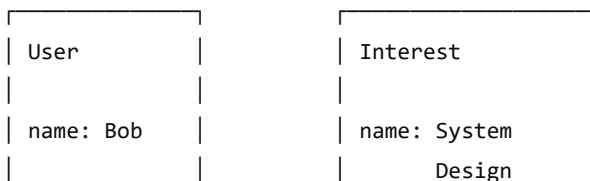
```
(systemDesign:Interest {name: "System Design"})
```

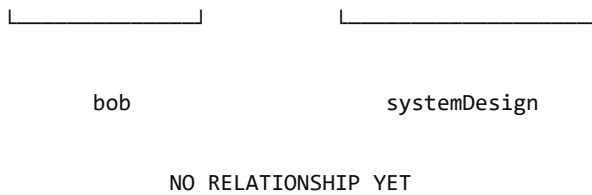
means:

Find an `Interest` whose name is `"System Design"`.

Current graph

At this point, conceptually:





This is important.

Query #5 only finds them. It does NOT connect them.

6. CREATE the Relationship

Now we finally connect them.

```

MATCH
  (bob:User {name: "Bob"}),
  (systemDesign:Interest {name: "System Design"})
CREATE
  (bob)-[:INTERESTED_IN]->(systemDesign)
RETURN bob, systemDesign;
  
```

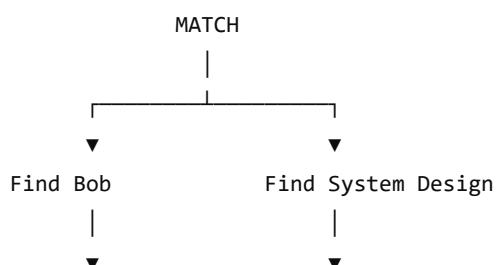
This is one of the most important Cypher patterns.

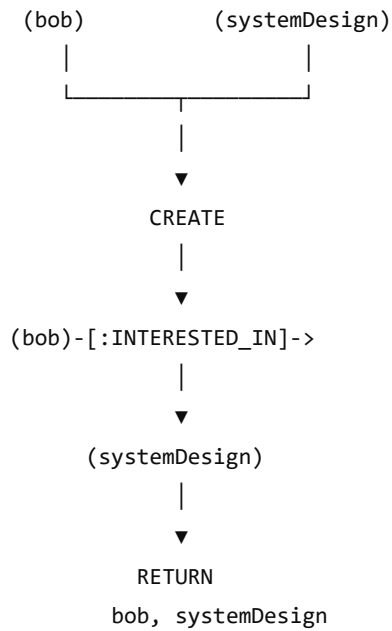
We are doing:

```

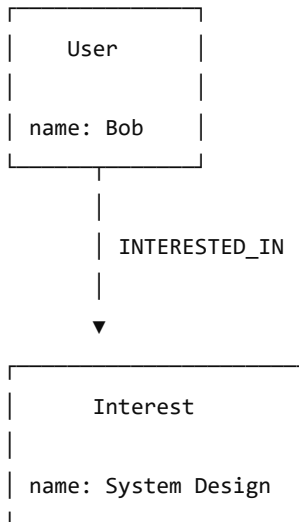
MATCH
  ↓
Find existing nodes
  ↓
CREATE
  ↓
Create relationship between them
  ↓
RETURN
  ↓
Show result
  
```

Step-by-step Flow





The graph becomes:

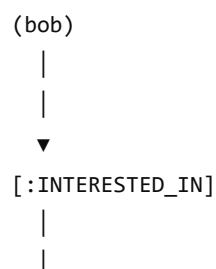


Understand the Relationship Syntax

This:

```
(bob)-[:INTERESTED_IN]->(systemDesign)
```

has three parts:



▼
(systemDesign)

Or:

SOURCE

|
| RELATIONSHIP

▼
TARGET

So:

Bob —INTERESTED_IN—> System Design

7. MATCH the Relationship and Return **u**

Now we get to the query from your screenshot:

```
MATCH u=()-[:INTERESTED_IN]->(a:Interest)
RETURN u;
```

This is extremely important because **u** is NOT a node here.

Let's break it carefully.

First look at the pattern

```
()-[:INTERESTED_IN]->(a:Interest)
```

This says:

Find something that has an **INTERESTED_IN** relationship pointing toward an **Interest**.

Visually:

```
unknown
  |
  | INTERESTED_IN
  ▼
(a:Interest)
```

The left node is:

()

We don't care what it is.

The right node is:

(a:Interest)

We specifically want an **Interest**.

But what is **u**?

Look carefully:

```
MATCH u=()-[:INTERESTED_IN]->(a:Interest)
```

The **u=** is placed **before the entire pattern**.

Therefore:

```
u
|
└── represents the ENTIRE PATH
```

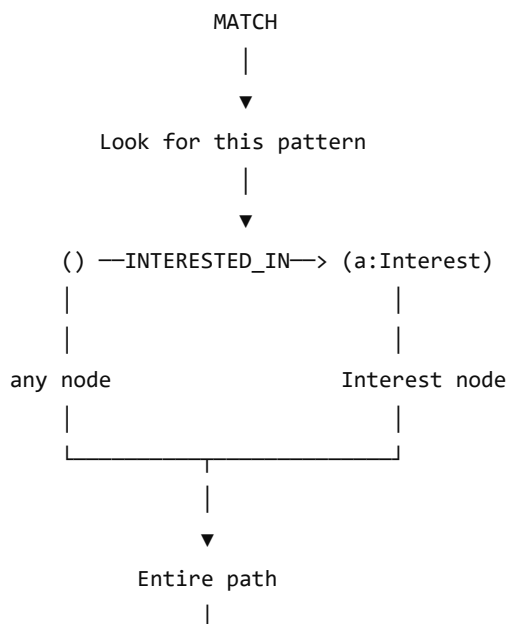
Not the starting node.

So:

```
u
|
▼
(Bob) —INTERESTED_IN→ (System Design)
```

u represents this complete path.

Flow of Query #7



```

▼
u
|
▼
RETURN
|
▼
u

```

So Neo4j returns:

Bob —INTERESTED_IN—> System Design

That's why your Neo4j screenshot shows:

Nodes: 2

Relationships: 1

Even though you wrote:

```
RETURN u
```

Because `u` represents the **path**, and that path contains:

2 nodes

+

1 relationship

8. Same MATCH Pattern, But Return `a`

Now:

```
MATCH u=()-[:INTERESTED_IN]->(a:Interest)
```

```
RETURN a;
```

The `MATCH` part is exactly the same.

Only the `RETURN` changed.

Query #7

```
MATCH u=()-[:INTERESTED_IN]->(a:Interest)
```

```
RETURN u;
```

Returns:

```
u
```

```
|
```

```
▼
```

Entire path

Bob —INTERESTED_IN—> System Design

Query #8

```
MATCH u=()-[:INTERESTED_IN]->(a:Interest)
```

```
RETURN a;
```

Returns:

```
a
```

```
|
```

```
▼
```

Only the Interest node

System Design

The Most Important Difference: **u** vs **a**

Look at this query:

```
MATCH u=()-[:INTERESTED_IN]->(a:Interest)
```

```
RETURN u;
```

There are **two variables**:

```
u
```

```
|
```

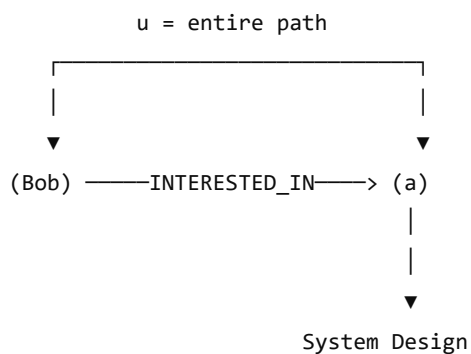
```
└─ Entire PATH
```

```
a
```

```
|
```

```
└─ Destination NODE
```

Graph:



Therefore:

RETURN u

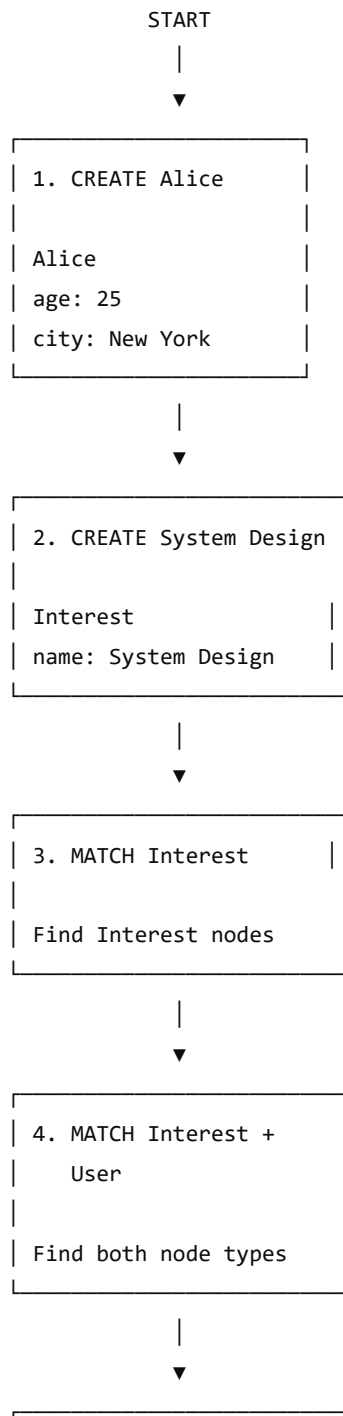
→ return the **path**

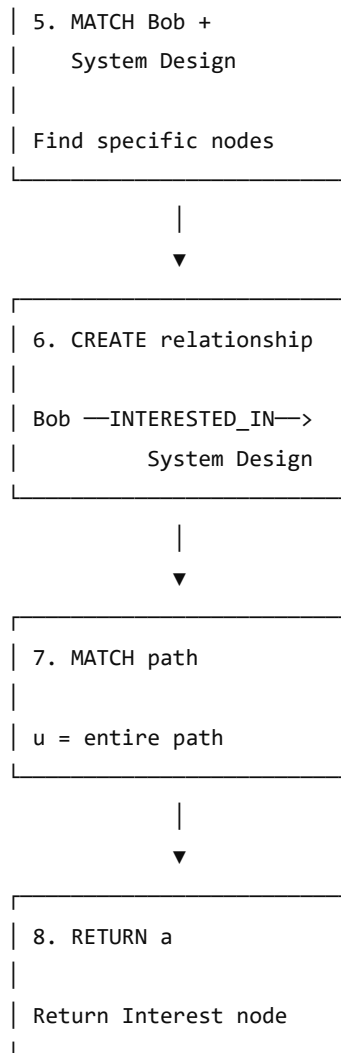
RETURN a

→ return the **Interest node**

All 8 Queries as One Big Flow

Now connect everything together.



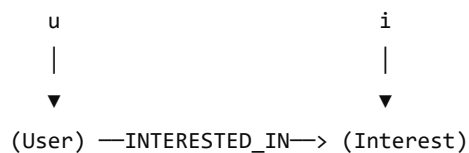


The Most Important Cypher Concept

When you see:

```
MATCH (u:User)-[:INTERESTED_IN]->(i:Interest)
RETURN u, i
```

think:



When you see:

```
MATCH path=(u:User)-[:INTERESTED_IN]->(i:Interest)
RETURN path
```

think:

path

|

▼

(User) —INTERESTED_IN—> (Interest)

When you see:

RETURN u

→ **User node**

When you see:

RETURN i

→ **Interest node**

When you see:

RETURN path

→ **Entire path**

That's the key to understanding the Cypher queries in your screenshot.

One small correction to remember

Your Query #1:

```
CREATE (aliceUser { name: "Alice", age: 25, city: "New York" });
```

creates an **unlabeled node**.

If you intend Alice to be a user, use:

```
CREATE (aliceUser:User {
  name: "Alice",
  age: 25,
  city: "New York"
});
```

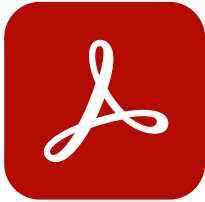
And Query #2 should be:

```
CREATE (systemDesign:Interest {
  name: "System Design"
});
```

5 key takeaways

1. **CREATE** creates nodes/relationships; **MATCH** finds existing graph data.
2. **(u:User {name: "Bob"})** means: find a **User** node whose **name** is Bob.

3. `(bob)-[:INTERESTED_IN]->(systemDesign)` creates a directed relationship.
4. `u=pattern` captures the entire pattern as a path variable.
5. In `MATCH u()-[:INTERESTED_IN]->(a:Interest)`, `u` = entire path, while `a` = only the Interest node.



Convert to PDF